```
import random

TOTAL_BANDWIDTH = 100
NUM_USERS = 5
POPULATION_SIZE = 20
GENERATIONS = 100
MUTATION_RATE = 0.1

demand = [30, 25, 40, 20, 35]
efficiency = [4, 5, 3, 6, 4]

def initialize_population():
    population = []

    for _ in range(POPULATION_SIZE):
        solution = [
            random.uniform(0, demand[i])
            for i in range(NUM_USERS)
        ]
        population.append(solution)
    return population

def check_constraints(solution):
    solution = [
        max(0, min(solution[i], demand[i]))
        for i in range(NUM_USERS)
    ]
    total = sum(solution)

    if total > TOTAL_BANDWIDTH:
        solution = [
            b * TOTAL_BANDWIDTH / total
            for b in solution
        ]
    return solution

def fitness(solution):
    solution = check_constraints(solution)
    throughput = sum(
        solution[i] * efficiency[i]
        for i in range(NUM_USERS)
    )
    utilization = (
        sum(solution) / TOTAL_BANDWIDTH
    )
    return throughput + 100 * utilization

def select_parent(population):
    candidates = random.sample(
        population, 3
    )
    return max(candidates, key=fitness)

def crossover(parent1, parent2):
    child = []

    for i in range(NUM_USERS):
        if random.random() < 0.5:
            child.append(parent1[i])
        else:
            child.append(parent2[i])
    return check_constraints(child)

def mutation(solution):
    child = solution.copy()

    if random.random() < MUTATION_RATE:
        i = random.randrange(NUM_USERS)
        child[i] = random.uniform(0, demand[i])
    return check_constraints(child)

def genetic_algorithm():
    population = initialize_population()
```

```
    for generation in range(GENERATIONS):
        population.sort(
            key=fitness, reverse=True
        )
        new_population = [population[0]]

        while len(new_population) < POPULATION_SIZE:

            parent1 = select_parent(population)
            parent2 = select_parent(population)
            child = crossover(parent1, parent2)
            child = mutation(child)
            child = check_constraints(child)
            new_population.append(child)
        population = new_population
    return max(population, key=fitness)

best = check_constraints(genetic_algorithm())

throughput = sum(
    best[i] * efficiency[i]
    for i in range(NUM_USERS)
)
utilization = (
    sum(best) / TOTAL_BANDWIDTH
) * 100
print("\n5G NETWORK RESOURCE ALLOCATION")
print("--------------------------------")

for i in range(NUM_USERS):
    print(
        f"User {i + 1}: {best[i]:.2f} MHz"
    )

print("--------------------------------")
print(f"Total Bandwidth: {sum(best):.2f} MHz")
print(f"Throughput: {throughput:.2f} Mbps")
print(f"Resource Utilization: {utilization:.2f}%")
print(f"Fitness Value: {fitness(best):.2f}")
```

```
5G NETWORK RESOURCE ALLOCATION
--------------------------------
User 1: 17.54 MHz
User 2: 23.65 MHz
User 3: 16.55 MHz
User 4: 17.31 MHz
User 5: 24.95 MHz
--------------------------------
Total Bandwidth: 100.00 MHz
Throughput: 441.71 Mbps
Resource Utilization: 100.00%
Fitness Value: 541.71
```

9/10/26

Genetic optimization Algorithm

Pseudocode

~~generation = 0;~~

1. Initialize a random population of Solution
2. Evaluate the fitness of each individual.
3. while Stopping Condition is not Satisfied

   Do,

   a) Select parents based on fitness.

   b) perform Crossover on selected parents to generate offsprings

   c) perform mutation on offspring with a small probability

   d) Evaluate the fitness of the offspring

   e) From a new population by replacing some or all of the old population

   End while

4. return the best Solution

5. End

5G - Network Resource Allocation

Allocate Bandwidth and Communication resources among users.

Maximize throughput and improve the resource utilization.

Pseudo Code

```
Begin.
    initialize random population of bandwidth
    allocation solutions
    Evaluate fitness of each solution
    while stopping condition is not satisfied
    do
        Select parent solutions based on fitness
        Apply crossover to generate offspring
        Apply mutation of off.spring
        check 5G Bandwidth and resource
    Constraints
        Evaluate fitness of offspring
        Select the best solutions to form
        the new population
    End while
        Display the best bandwidth allocation
End.
```

output:

5 eg Network Resource Allocation.

User 1: 17.54 MHz

User 2 : 23.65 MHz

User 3 : 16.55 MHz

User 4 : 17.31 MHZ

User 5: 24.95 MHZ

Total Bandwidth: 100.00 MHz

Throughput : 441.71 Mbps

Resource Utilization : 100.00%.

Fitness value : 541.71

O/P Seen
09/10/26